

14_digitos

May 20, 2026

```
[4]: import matplotlib.pyplot as plt
import torch #usada para machine learning y deep learning.
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import numpy as np
```

1 Manejo de datos

El manejo de datos suele representar una parte importante del código necesario para realizar aprendizaje automático. Antes de entrenar un modelo, necesitamos preparar los datos en un formato compatible con el entrenamiento y la evaluación.

Normalmente no entrenamos con todo el conjunto de datos al mismo tiempo, porque tardaría demasiado. En su lugar, entrenamos y evaluamos por lotes (*batches*), donde cada lote contiene una pequeña muestra de los datos. El entrenamiento se realiza en múltiples iteraciones, usando una muestra aleatoria diferente en cada iteración.

El objetivo principal de construir un modelo (de clasificación o regresión) es hacer predicciones sobre datos no vistos. A esto se le llama generalización. Por ello, debemos dividir los datos en entrenamiento y prueba. Si accidentalmente entrenamos con los datos de prueba, el modelo parecerá funcionar mejor, pero no sería una evaluación real de su capacidad de generalización.

Como ejemplo, utilizaremos el conjunto de datos MNIST, que contiene imágenes de números escritos a mano junto con sus etiquetas correspondientes. La tarea consiste en predecir qué número representa una imagen no vista y así entrenar un modelo de reconocimiento de escritura manuscrita.

Este es un conjunto de datos estándar disponible en PyTorch y ya viene dividido en entrenamiento y prueba.

```
[5]: conjunto_entrenamiento = datasets.MNIST('files/', train=True, download=True,
                                             transform=transforms.Compose([
                                                 transforms.ToTensor(),
                                                 transforms.Normalize(
                                                     (0.1307,), (0.3081,))
                                             ]))
conjunto_prueba = datasets.MNIST('files/', train=False, download=True,
                                  transform=transforms.Compose([
                                      transforms.ToTensor(),
                                      transforms.Normalize(
```

```
(0.1307,), (0.3081,))  
]))
```

```
100.0%  
100.0%  
100.0%  
100.0%
```

Esto nos proporciona los conjuntos de entrenamiento y prueba de imágenes (datos) junto con sus etiquetas correspondientes (targets).

```
[6]: conjunto_entrenamiento
```

```
[6]: Dataset MNIST  
      Number of datapoints: 60000  
      Root location: files/  
      Split: Train  
      StandardTransform  
      Transform: Compose(  
          ToTensor()  
          Normalize(mean=(0.1307,), std=(0.3081,))  
      )
```

```
[7]: print(conjunto_entrenamiento.data.shape)  
      print(conjunto_prueba.data.shape)  
      print(conjunto_entrenamiento.targets.shape)  
      print(conjunto_prueba.targets.shape)  
      print(conjunto_prueba.data.dtype)  
      print(conjunto_prueba.targets.dtype)
```

```
torch.Size([60000, 28, 28])  
torch.Size([10000, 28, 28])  
torch.Size([60000])  
torch.Size([10000])  
torch.uint8  
torch.int64
```

Observa la forma (*shape*) y el tipo de dato de los conjuntos.

Para pasar los datos al modelo, utilizaremos la función `DataLoader`.

```
[8]: tamaño_lote_entrenamiento = 64  
      tamaño_lote_prueba = 1000  
  
      cargador_entrenamiento = DataLoader( conjunto_entrenamiento,   
          ↪ batch_size=tamaño_lote_entrenamiento, shuffle=True, )  
  
      cargador_prueba = DataLoader( conjunto_prueba, batch_size=tamaño_lote_prueba,   
          ↪ shuffle=True)
```

Los DataLoader pueden usarse para generar una submuestra iterando sobre ellos.

```
[9]: ejemplos = enumerate(cargador_entrenamiento)
    indice_lote, (datos_ejemplo, etiquetas_ejemplo) = next(ejemplos)
    print(datos_ejemplo.shape)
    print(datos_ejemplo.dtype)
    print(etiquetas_ejemplo.shape)
    print(etiquetas_ejemplo.dtype)
```

```
torch.Size([64, 1, 28, 28])
torch.float32
torch.Size([64])
torch.int64
```

Los ejemplos de prueba tienen 1000 muestras de 28x28 píxeles. Observa que DataLoader ha añadido una dimensión y cambiado el tipo de dato para hacerlo compatible con los modelos de PyTorch.

Los ejemplos de entrenamiento tienen la misma forma, pero solo 64 muestras.

Podemos mostrar los primeros ejemplos para ver cómo se ven y compararlos con sus etiquetas.

```
[10]: ejemplos = enumerate(cargador_entrenamiento)
    indice_lote, (datos_ejemplo, etiquetas_ejemplo) = next(ejemplos)
    figura, ejes = plt.subplots(nrows=2, ncols=4, figsize=(10, 5))
    ejes = ejes.flatten()
    for eje, imagen, etiqueta in zip(ejes, datos_ejemplo, etiquetas_ejemplo):
        eje.set_axis_off()
        eje.imshow(imagen[0], cmap=plt.cm.gray_r, interpolation="nearest")
        eje.set_title("Entrenamiento: %i" % etiqueta)
```

Entrenamiento: 4



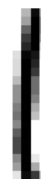
Entrenamiento: 9



Entrenamiento: 2



Entrenamiento: 1



Entrenamiento: 4



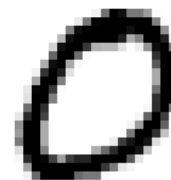
Entrenamiento: 0



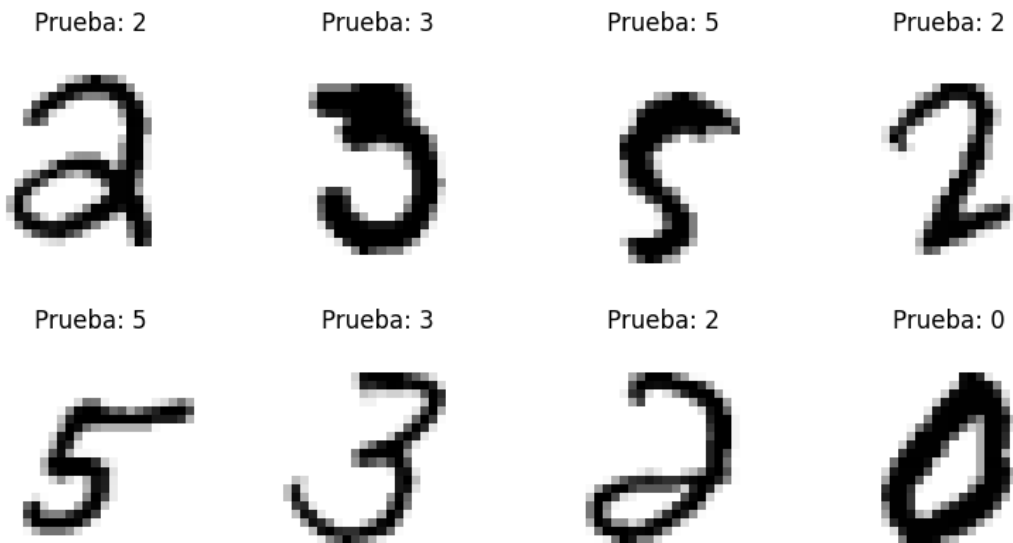
Entrenamiento: 3



Entrenamiento: 0



```
[11]: ejemplos = enumerate(cargador_prueba)
indice_lote, (datos_ejemplo, etiquetas_ejemplo) = next(ejemplos)
figura, ejes = plt.subplots(nrows=2, ncols=4, figsize=(10, 5))
ejes = ejes.flatten()
for eje, imagen, etiqueta in zip(ejes,datos_ejemplo,etiquetas_ejemplo):
    eje.set_axis_off()
    eje.imshow(imagen[0], cmap=plt.cm.gray_r, interpolation="nearest")
    eje.set_title("Prueba: %i" % etiqueta)
```



2 Ahora construyamos algunos modelos

Primero necesitamos cargar algunas herramientas adicionales de PyTorch.

```
[12]: import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

Aplanaremos las imágenes para que cada una sea un tensor 1D (¿es una buena idea para imágenes?). Por ello, queremos saber cuántos elementos tendrá ese tensor 1D.

```
[13]: tamaño_imagen = conjunto_entrenamiento.data.shape[1:]
tamaño_capa_entrada = np.prod(tamaño_imagen)
tamaño_capa_entrada
```

```
[13]: np.int64(784)
```

Como trabajamos en base 10, sabemos que existen 10 clases posibles a las que puede pertenecer cada imagen.

```
[14]: numero_clases = 10
```

Ahora definiremos una clase que describirá nuestro modelo.

Heredará métodos y propiedades de `nn.Module`, que incluye muchas utilidades ya implementadas.

```
[15]: class RedNeuronal(nn.Module): # Hereda de nn.Module, que es la clase base para
    todos los módulos de redes neuronales.
    def __init__(self): # Se ejecuta cuando se inicializa la clase.
        super().__init__() # Inicializa los métodos y propiedades de la clase
        base Module.

        # Las capas se definen con el enfoque secuencial, lo que simplifica la
        estructura.
        self.clasificador = nn.Sequential(
            nn.Linear(tamano_capa_entrada, 500), # Capa oculta
            nn.ReLU(),
            nn.Linear(500, 500), # Capa oculta
            nn.Linear(500, numero_clases), # Capa de salida, porque es la
            última capa.
        )

    def forward(self, entrada): # Este método se usa para la propagación hacia
    adelante de la red.
        entrada = entrada.view(-1,tamano_capa_entrada) # Primero se
        reestructuran todas las imágenes.
        entrada = self.clasificador(entrada) # Después se aplican
        las capas del clasificador.
        return F.log_softmax(entrada,dim=0) # Finalmente se devuelven
        probabilidades para cada clase usando la función softmax.
```

A continuación se muestra un modelo idéntico, pero definido sin usar el enfoque secuencial.

```
[16]: class RedNeuronal(nn.Module): # Hereda de nn.Module, que es la clase base para
    todos los módulos de redes neuronales.
    def __init__(self): # Se ejecuta cuando se inicializa la clase.
        super().__init__() # Inicializa los métodos y propiedades de la clase
        base Module.

        # Las capas se definen individualmente y luego se llaman por separado
        en el método forward.
        self.capa_lineal_1 = nn.Linear(tamano_capa_entrada, 500) # Capa oculta
        self.capa_lineal_2 = nn.Linear(500, numero_clases) # Capa de salida,
        porque es la última capa.
```

```

def forward(self, entrada): # Este método se usa para la propagación hacia
↪adelante de la red.
    entrada = entrada.view(-1,tamano_capa_entrada) # Primero se
↪reestructuran todas las imágenes.
    entrada = self.capa_lineal_1(entrada) # Después se aplica
↪la primera capa.
    entrada = self.capa_lineal_2(entrada) # Después se aplica
↪la segunda capa.
    return F.log_softmax(entrada,dim=0) # Finalmente se devuelven
↪probabilidades para cada clase usando la función softmax.

```

Ahora podemos inicializar el modelo.

```
[17]: modelo = RedNeuronal()
```

Para realizar una propagación hacia adelante (*forward pass*), simplemente pasamos un lote de muestras al modelo (esto usa el método `forward()`).

```

[18]: indice_lote, (datos, etiqueta_objetivo) =
↪next(enumerate(cargador_entrenamiento))
salida = modelo(datos)
salida[0]

```

```
[18]: tensor([-4.1985, -4.0763, -3.8724, -4.0286, -4.7756, -3.8891, -4.1058, -4.0642,
-4.2789, -3.9889], grad_fn=<SelectBackward0>)
```

Esto nos da las probabilidades de que cada imagen pertenezca a cada clase.

Ahora debemos encontrar el número con mayor probabilidad para cada imagen y compararlo con las etiquetas reales.

```
[19]: _, predicho = torch.max(salida, 1)
predicho
```

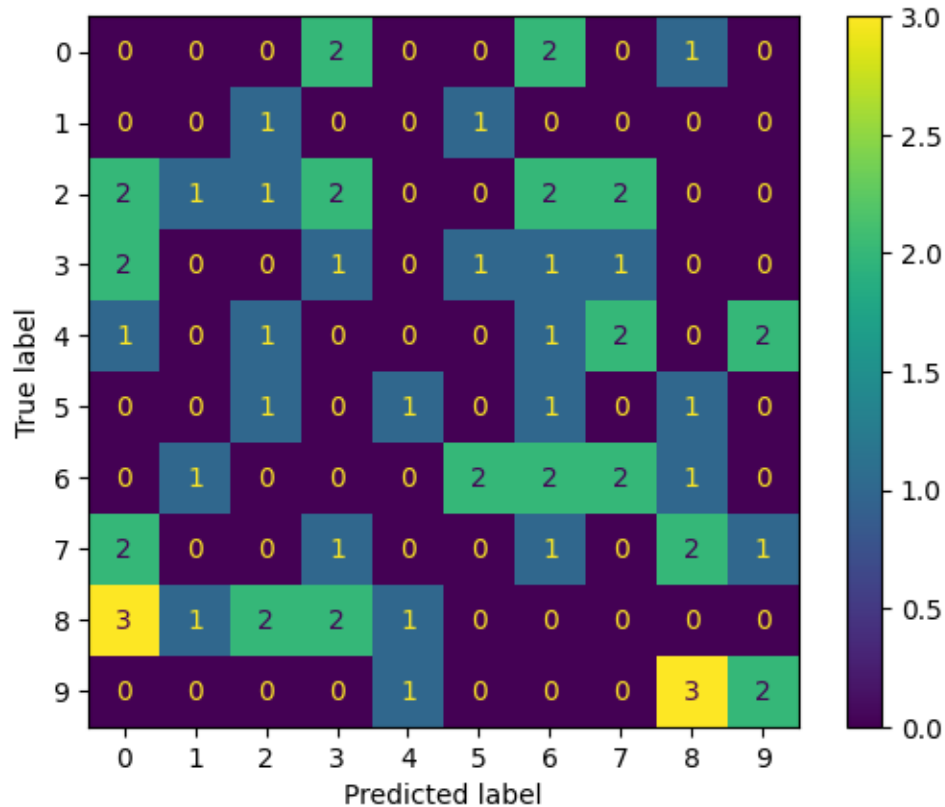
```
[19]: tensor([2, 7, 5, 8, 5, 4, 0, 0, 0, 6, 8, 0, 3, 0, 2, 3, 6, 2, 0, 0, 6, 5, 7, 2,
6, 9, 7, 0, 1, 6, 9, 8, 9, 0, 1, 4, 7, 6, 0, 3, 1, 3, 8, 8, 9, 8, 6, 6,
3, 3, 8, 5, 6, 3, 6, 2, 3, 7, 2, 7, 4, 9, 7, 8])
```

Scikit-learn tiene una función útil para evaluar qué tan buenas son las predicciones mediante una matriz de confusión.

```

[20]: from sklearn import metrics
visualizacion = metrics.ConfusionMatrixDisplay.
↪from_predictions(etiqueta_objetivo, predicho)

```



Como era de esperarse, una red neuronal completamente aleatoria y sin entrenar no se ajusta bien a nuestros datos. ¡Necesitamos entrenarla!

Usaremos la pérdida de entropía cruzada (*cross entropy*), común en clasificadores, y el optimizador Adam, un algoritmo adaptativo muy utilizado en redes neuronales.

```
[21]: %%time
criterio = nn.NLLLoss()
optimizador = optim.Adam(modelo.parameters(), lr=3e-4) # Se usa una tasa de
↳ aprendizaje de 3e-4: ni demasiado grande ni demasiado pequeña.

# Se itera sobre cada lote; es decir, una pasada completa por todo el conjunto
↳ de datos.
# Esto se conoce como una época de entrenamiento.
for indice_lote, (datos, etiqueta_objetivo) in
↳ enumerate(cargador_entrenamiento):

    modelo.train() # IMPORTANTE: asegurar que el modelo esté en modo de
↳ entrenamiento.
    optimizador.zero_grad() # Reinicia los gradientes antes del cálculo.
    salida = modelo(datos) # Propagación hacia adelante del modelo.
```

```

    perdida = criterio(salida, etiqueta_objetivo) # Calcula la función de
    ↪pérdida a partir de la propagación hacia adelante.
    perdida.backward() # Usa la pérdida para realizar retropropagación y
    ↪obtener los gradientes de los pesos.
    optimizador.step() # Actualiza los pesos del modelo para mejorar el ajuste.

```

CPU times: total: 1min 37s

Wall time: 25.2 s

Ahora evaluemos recorriendo el conjunto de prueba y comparando contra las etiquetas reales.

```

[22]: perdida_prueba = 0
    correctos = 0
    arreglo_predicciones = np.array([])
    arreglo_etiquetas = np.array([])

    modelo.eval() # IMPORTANTE: cambiar a modo de evaluación, ya que modifica
    ↪algunos comportamientos.
    with torch.no_grad(): # Desactiva el cálculo de gradientes porque ya no se está
    ↪entrenando.
        for datos, etiqueta_objetivo in cargador_prueba:
            salida = modelo(datos)
            prediccion = salida.data.max(1, keepdim=True)[1]
            correctos += prediccion.eq(etiqueta_objetivo.data.view_as(prediccion)).
            ↪sum()
            arreglo_predicciones = np.append(arreglo_predicciones, prediccion)
            arreglo_etiquetas = np.append(arreglo_etiquetas, etiqueta_objetivo)

    correctos/len(cargador_prueba.dataset)

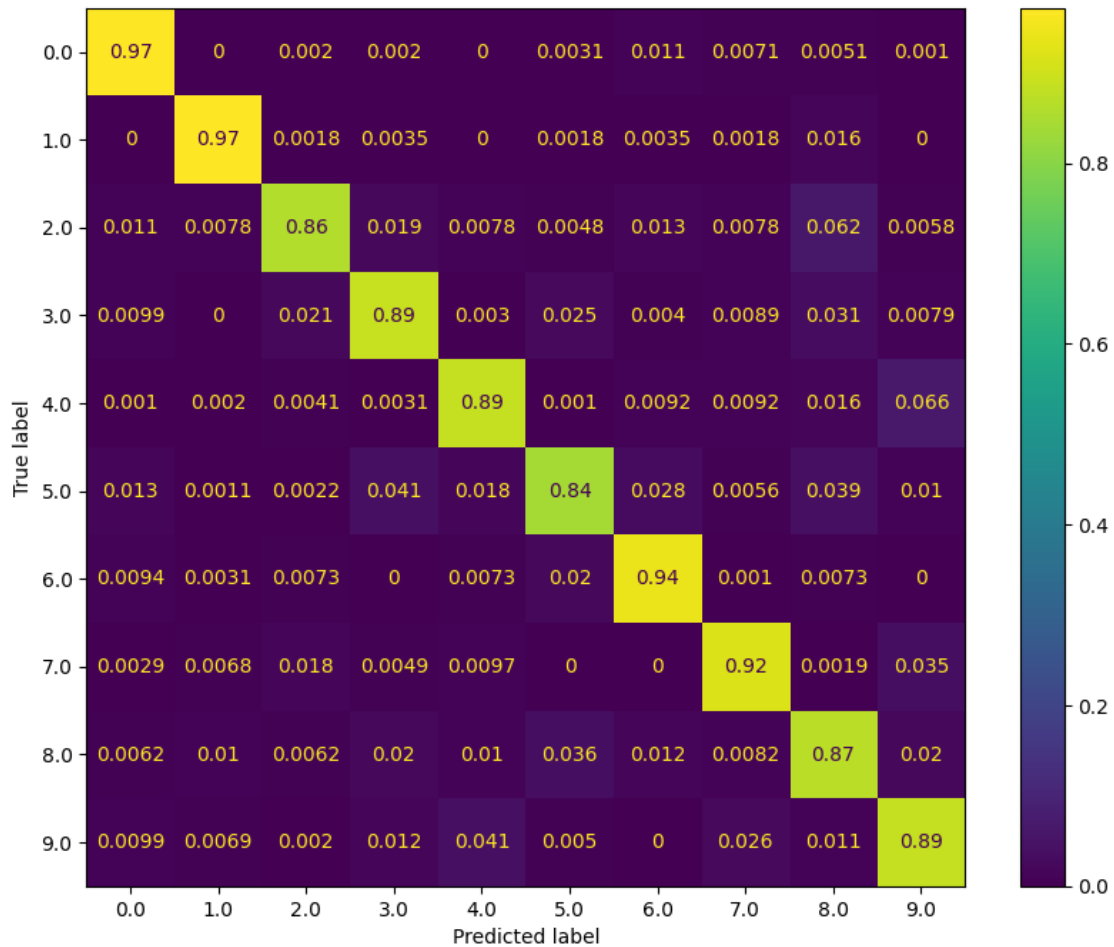
```

[22]: tensor(0.9058)

```

[23]: visualizacion = metrics.ConfusionMatrixDisplay.
    ↪from_predictions(arreglo_etiquetas, arreglo_predicciones, normalize='true')
    figura = plt.gcf()
    figura.set_figheight(8)
    figura.set_figwidth(10)

```

¡Más del 90% de exactitud y la matriz de confusión se ve bien!

3 Reto: ¿puedes superar la exactitud anterior?

¿Cómo podrías hacerlo? - ¿Cambiando la arquitectura del modelo? - ¿Usa funciones de activación diferentes? (por ejemplo `nn.ReLU`) - ¿Modificando el optimizador? - ¿Entrenando durante múltiples épocas?

[24]: `nn.ReLU?`

Init signature: `nn.ReLU(inplace: bool = False) ->`

`None`

Docstring:

Applies the rectified linear unit function element-wise.

:math: $\text{ReLU}(x) = \max(0, x)$

Args:

inplace: can optionally do the operation in-place. Default: ``False``

Shape:

- Input: $(*)$, where $*$ means any number of dimensions.
- Output: $(*)$, same shape as the input.

.. image:: ../scripts/activation_images/ReLU.png

Examples::

```
>>> m = nn.ReLU()
>>> input = torch.randn(2)
>>> output = m(input)
```

An implementation of CReLU - <https://arxiv.org/abs/1603.05201>

```
>>> m = nn.ReLU()
>>> input = torch.randn(2).unsqueeze(0)
>>> output = torch.cat((m(input), m(-input)))
```

Init docstring: Initialize internal Module state, shared by both nn.Module and ScriptModule.

File: c:\users\jessan\desktop\almacenes_mineria_datos\github\redes_neuronales\.venv\lib\site-packages\torch\nn\modules\activation.py

Type: type

Subclasses: ReLU6

3.1 Define tu modelo

```
[25]: class RedNeuronal(nn.Module): # Hereda de nn.Module, que es la clase base para
    ↪ todos los módulos de redes neuronales.
    def __init__(self): # Se ejecuta cuando se inicializa la clase.
        super().__init__() # Inicializa los métodos y propiedades de la clase
    ↪ base Module.

        # Las capas se definen con el enfoque secuencial, lo que simplifica la
    ↪ estructura.
        self.clasificador = nn.Sequential(
            nn.Linear(tamano_capa_entrada, 500), # Capa oculta
            nn.BatchNorm1d(500),
            nn.ReLU(inplace=True),
            nn.Linear(500, 500), # Capa oculta
            nn.BatchNorm1d(500),
            nn.ReLU(inplace=True),
            nn.Linear(500, numero_clases), # Capa de salida, porque es la
    ↪ última capa.
        )
```

```

def forward(self, entrada): # Este método se usa para la propagación hacia
↪adelante de la red.
    entrada = entrada.view(-1,tamano_capa_entrada) # Primero se
↪reestructuran todas las imágenes.
    entrada = self.clasificador(entrada) # Después se aplican
↪las capas del clasificador.
    return F.log_softmax(entrada,dim=0) # Finalmente se devuelven
↪probabilidades para cada clase usando la función softmax.

```

3.2 Define tu función de pérdida, inicializa tu modelo y un optimizador

```

[26]: modelo = RedNeuronal()
criterio = nn.NLLLoss()
optimizador = optim.Adam(modelo.parameters(), lr=3e-4) # Se usa una tasa de
↪aprendizaje de 3e-4: ni demasiado grande ni demasiado pequeña.

```

3.3 Entrena tu modelo

Itera sobre cada lote; una pasada completa por todo el conjunto de entrenamiento se conoce como una época de entrenamiento.

```

[27]: %%time

# Se itera sobre cada lote; es decir, una pasada completa por todo el conjunto
↪de datos.
# Esto se conoce como una época de entrenamiento.
for indice_lote, (datos, etiqueta_objetivo) in
↪enumerate(cargador_entrenamiento):

    modelo.train()
    optimizador.zero_grad()
    salida = modelo(datos)
    perdida = criterio(salida, etiqueta_objetivo)
    perdida.backward()
    optimizador.step()

```

CPU times: total: 2min 7s

Wall time: 34.1 s

3.4 Evalúa el rendimiento de tu modelo

Itera sobre cada lote del conjunto de prueba para obtener la exactitud y generar arreglos con etiquetas reales y predichas.

```
[28]: exactitud_prueba = []
exactitud_entrenamiento = []
# Evaluar el modelo:
modelo.eval()
perdida_prueba = 0
correctos = 0
arreglo_predicciones = np.array([])
arreglo_etiquetas = np.array([])
with torch.no_grad():
    for datos, etiqueta_objetivo in cargador_prueba:
        salida = modelo(datos)
        prediccion = salida.data.max(1, keepdim=True)[1]
        correctos += prediccion.eq(etiqueta_objetivo.data.view_as(prediccion)).
        ↪sum()
        arreglo_predicciones = np.append(arreglo_predicciones, prediccion)
        arreglo_etiquetas = np.append(arreglo_etiquetas, etiqueta_objetivo)

correctos/len(cargador_prueba.dataset)
```

[28]: tensor(0.9727)

3.5 Grafica la matriz de confusión

Usa los arreglos de etiquetas reales y predichas para graficar la matriz de confusión. Normaliza por los valores reales para mostrar qué clases se predicen mejor.

```
[29]: visualizacion = metrics.ConfusionMatrixDisplay.
        ↪from_predictions(arreglo_etiquetas, arreglo_predicciones, normalize='true')
figura = plt.gcf()
figura.set_figheight(8)
figura.set_figwidth(10)
```

